

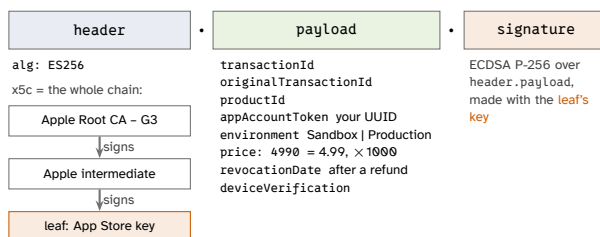
In one line: A StoreKit 2 purchase ends in a **signed letter**, not a callback: each Transaction is a **JWS** the App Store signs and StoreKit checks *on the device*, against the certificate chain packed inside it. Your part is **verify** → **grant** → **finish()** — with an ear open from launch (Transaction.updates) for all that is bought, renewed or refunded while you are not looking.

Why it looks like this



- **2012:** on iOS ≤ 5.1 an app asked Apple's server "is this receipt valid?" itself. A DNS redirect plus a user-installed CA put an impostor there that said yes, and generic receipts could be replayed. Fix: ask from *your* server; iOS 6 closed it.
- **StoreKit 2 asks nobody:** the chain to Apple's root travels *inside* each JWS, so the check runs offline, and deviceVerification = SHA-384(nonce + this device's ID) ties it to one device.
- The old PKCS#7/ASN.1 receipt file is deprecated: Transaction and AppTransaction (iOS 16, the app's own purchase) replace it.

What a transaction really is



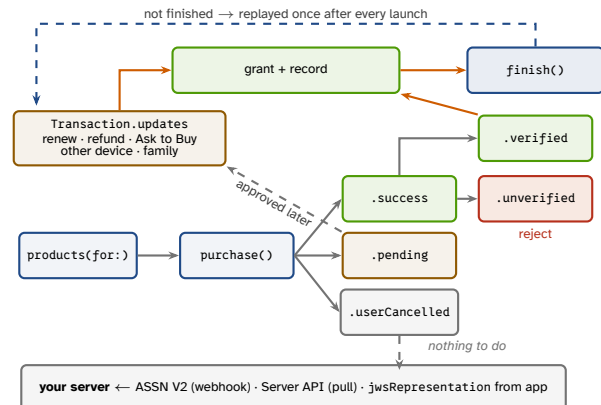
How it flows

- **Products live in the store:** Product.products(for:) answers an *empty array*, *not an error*, when IDs or the Paid Apps agreement in App Store Connect are wrong.
- **Owning:** currentEntitlements = latest transaction per non-consumable, live subscription (subscribed or in grace) and non-renewing sub; refunded ones vanish.
- **One plan per subscription group:** upgrade bites now, downgrade at the next renewal.
- **Refunds:** asked for in-app since iOS 15 (beginRefundRequest). Apple sends CONSUMPTION_REQUEST and weighs your "how much was used" if it comes within 12 hours; then REFUND and a revocationDate.
- **Tests:** a .storekit file (2020) runs offline; the sandbox is real servers on fast time. App Review buys in the sandbox with your *production* build: old servers retried on status 21007, a JWS just says environment.

Example: both roads, one handler

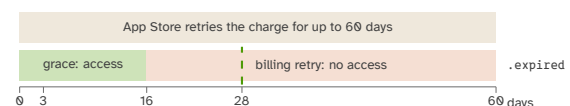
```
// App init: one listener for the app's lifetime
let ear = Task.detached {
  for await r in Transaction.updates { await handle(r) } }
func buy(_ p: Product) async throws {
  if case .success(let r) = try await p.purchase() {
    await handle(r) // .pending -> arrives on updates
  }
}
func handle(_ r: VerificationResult<Transaction>) async {
  guard case .verified(let t) = r else { return } // reject
  await grant(t) // durable first ...
  await t.finish() // ... then "delivered"
}
```

One purchase, two roads in



Ask to Buy expires: not approved in 24 h, the child asks again. Server notifications carry the same JWS. Device decides UX, server decides truth.

When a renewal charge fails



- Grace lasts 3, 16 or 28 days (weekly: 3 or 6), the sub still in currentEntitlements; in billing retry it drops out.
- Grace is money too: recovered in it, the *days of paid service* never break — a year of them in one group lifts your share from 70% to 85%. Back within 60 days the count goes on; later, it restarts.

Traps

- ! Trusting .unverified — *the* StoreKit security bug.
- ! No finish() → double grants; finishing *before* recording → a lost purchase.
- ! No updates listener from launch → missed renewals, refunds, approvals.
- ! Counting coins in currentEntitlements — consumables never appear there. Coins are your own ledger.
- ! Granting on .pending. Trusting sandbox timing.
- ! AppStore.sync() at launch — a sign-in prompt. Entitlements sync by themselves; sync() is for the Restore button.
- ! Booking revenue from the JWS price — App Store Connect reports are the record.

Sandbox time (default rate)

1 week	1 month	2 months	3 months	6 months	1 year
3 min	5 min	10 min	15 min	30 min	1 h

Renews up to 12 times, then stops. Billing retry 10 min, grace period 3–5 min. Rate is per sandbox account (3 min · 5 · 30 · 1 h) in App Store Connect.

Remember

"V-G-F, and keep ears open": Verify, Grant, Finish — and Transaction.updates runs from launch.